Accessing hardware devices



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



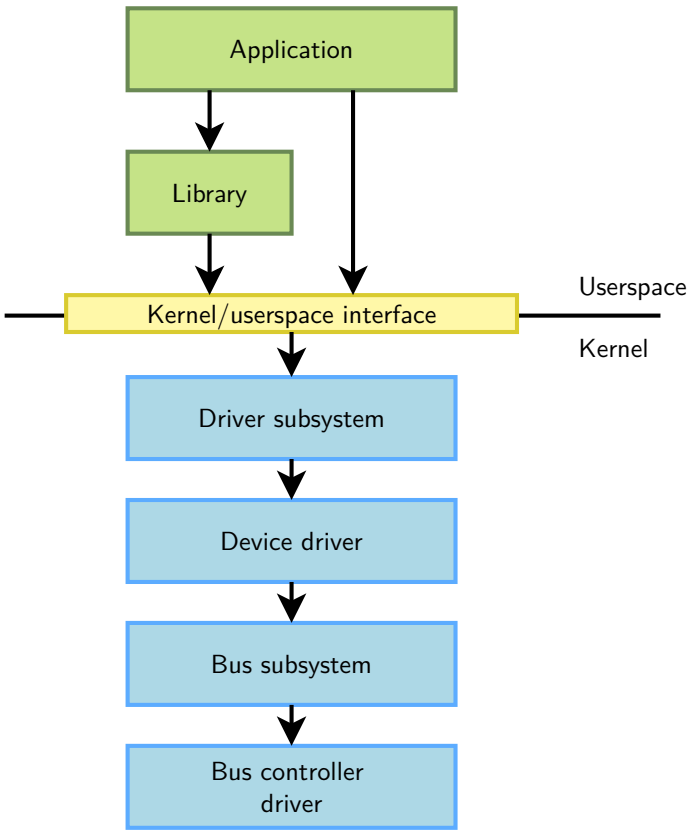
Kernel drivers



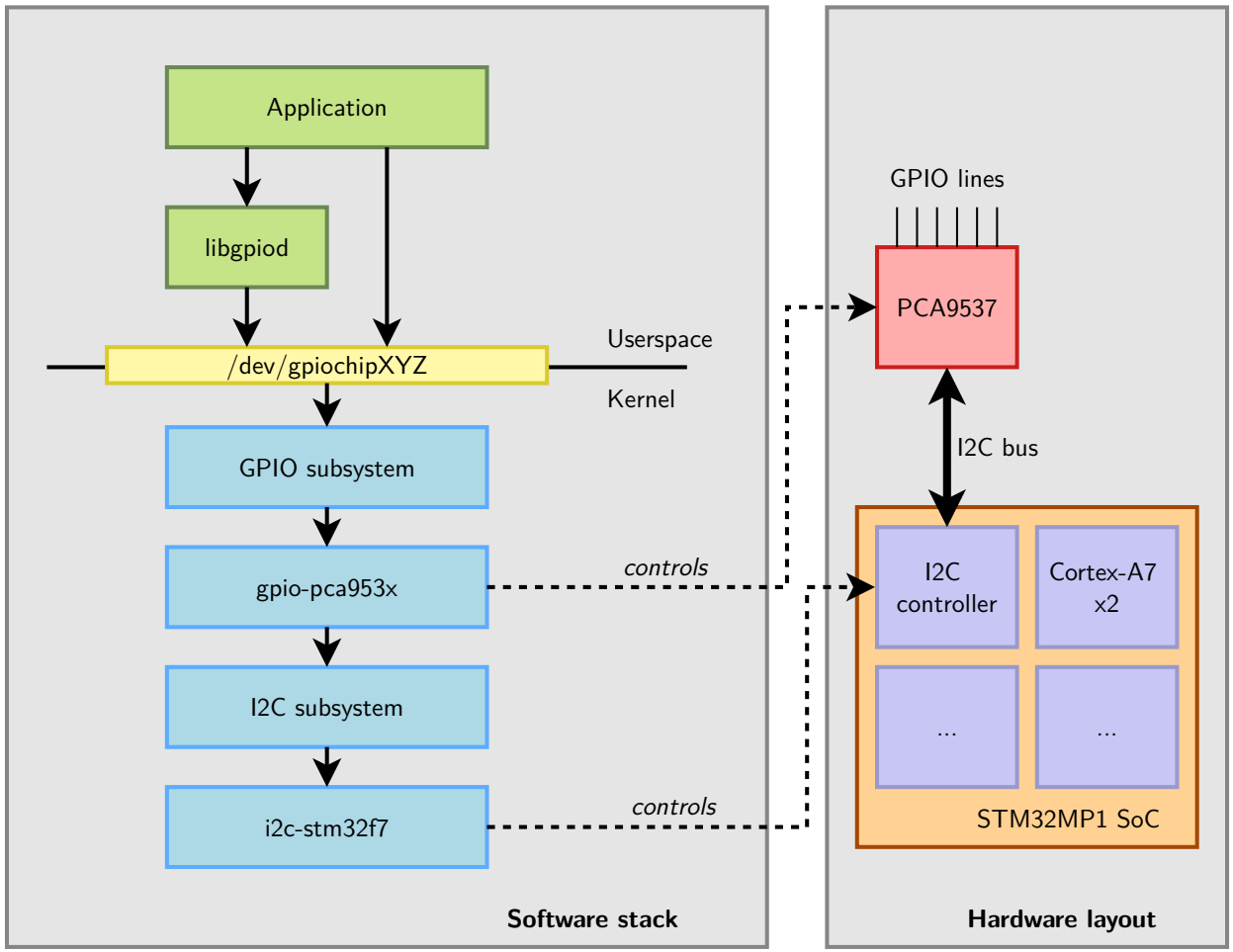
Typical software stack for hardware access

From the bottom to the top:

- A *bus controller driver* in the kernel drives an I2C, SPI, USB, PCI controller
- A *bus subsystem* provides an API for drivers to access a particular type of bus: I2C, SPI, PCI, USB, etc.
- A *device driver* in the kernel drives a particular device connected to a given bus
- A *driver subsystem* exposes features of certain class of devices, through a standard *kernel/user-space interface*
- An application can access the device through this standard *kernel/user-space interface* either directly or through a library.



Stack illustrated with a GPIO expander





Standardized user-space interface

- Strong advantage of kernel drivers: they expose a standard *kernel to user-space interface*
- All devices of the same class (e.g GPIO controllers) will expose the same *kernel to user-space interface*
- Applications don't have to know the details of the GPIO controller, they just need to know the standard user-space interface valid for all GPIO controllers
- Applications can use existing open-source libraries that leverage this standard user-space interface
- Such kernel drivers can also be used internally inside the kernel, for example if one driver needs to control a GPIO directly (reset signal, interrupt signal, etc.)



Numerous kernel subsystems for device classes

- Networking stack for Ethernet, WiFi, CAN, 802.15.4, etc.
- GPIO
- Video4Linux for camera, video encoders/ decoders
- DRM for display controllers, GPU
- ALSA for audio
- IIO for ADC, DAC, gyroscopes, sensors, and more
- MTD for flash memory
- PWM
- Input for keyboard, mouse, touchscreen, joystick
- Watchdog
- RTC for real-time clocks
- remoteproc for auxiliary processors
- crypto for cryptographic accelerators
- hwmon for hardware monitoring sensors
- block layer for block storage

and many more

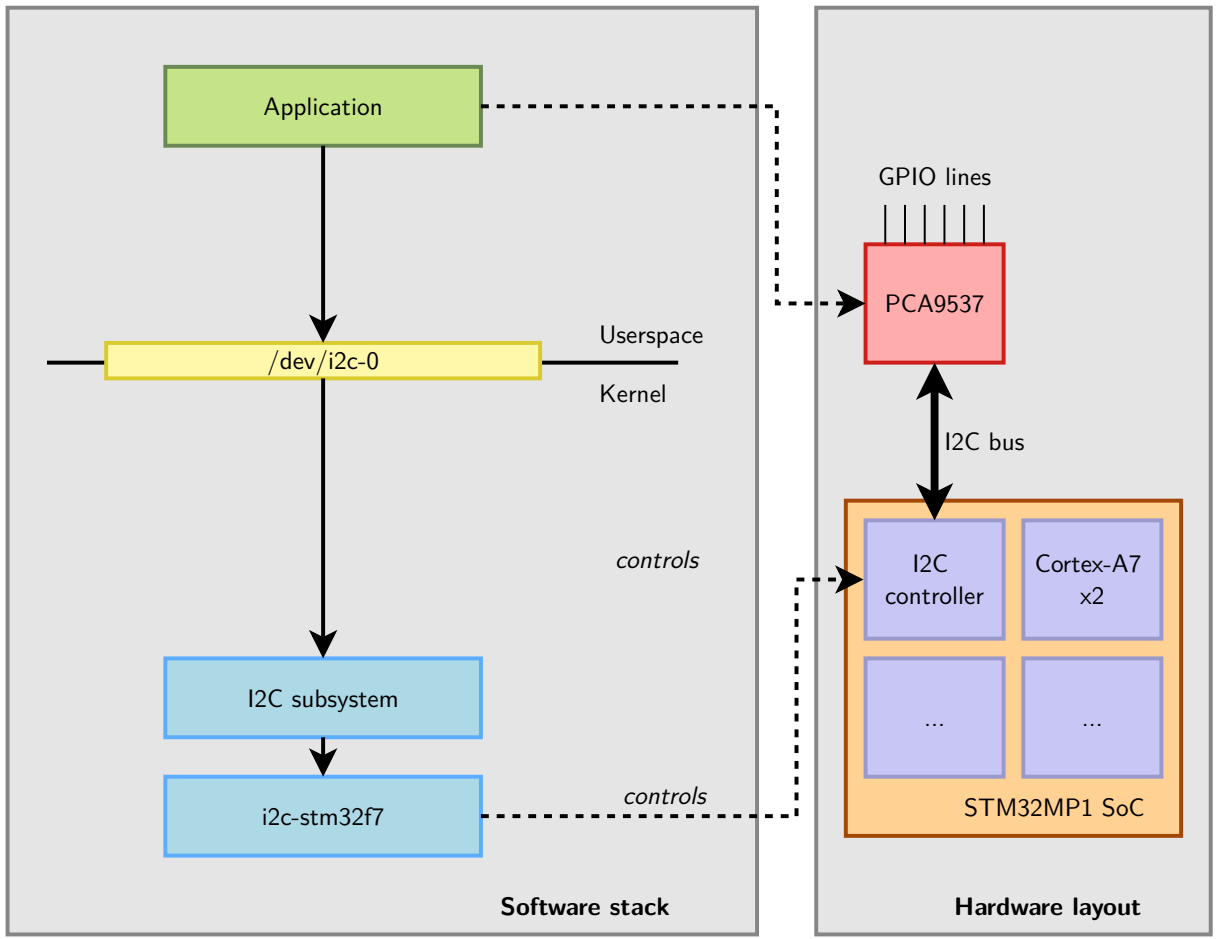


Accessing devices directly from user-space

- Even though device drivers in the kernel are preferred, it is also possible to access devices directly from user-space
- Especially useful for very specific devices that do not fit in any existing kernel subsystems
- The kernel provides the following mechanisms, depending on the bus:
  - I2C: i2c-dev
  - SPI: spidev
  - Memory-mapped: UIO
  - USB: `/dev/bus/usb`, through libusb
  - PCI: sysfs entries for PCI

Accessing devices directly from user-space: GPIO example

This diagram shows what's not recommended to do → for a GPIO controller, a kernel driver is preferred





What can go wrong with a user-space driver?

- You write your GPIO driver in user-space: other kernel drivers cannot use GPIOs from this GPIO controller
  - Other devices that use GPIO signals from this controller for reset, interrupt, etc. cannot control/configure those signals
  - Your application is less portable: it will take many changes to support another type of GPIO controller.
- You write your touchscreen driver in user-space: the standard Linux graphics stack components cannot use your touchscreen
- You write your network driver in user-space
  - You can probably send/receive packets
  - But you cannot leverage the Linux kernel networking stack for IP, TCP, UDP, etc.
  - And none of the Linux networking applications can use your network device



Upstream drivers vs. out-of-tree drivers

- The *upstream* Linux kernel contains thousands of drivers
  - This is the best place to look for drivers
  - Drivers have been reviewed and approved by the community
  - They comply with standard interfaces
- Vendor kernels often include additional drivers, directly in the kernel tree
- Device vendors sometimes also provide *out of tree drivers*
  - Their source code is provided separately from the Linux kernel tree
  - Quality is often dubious
  - Compatibility issues when updating to newer kernel releases
  - Not always use standard user-space interfaces
  - Example: https://github.com/lwfinger/rtl8723ds
  - Avoid them when possible!



Finding Linux kernel drivers

- `grep` in the Linux kernel tree is your *best friend*
  - For I2C, SPI and memory-mapped devices, matching of the driver is done based on the device name → *grep* for variants of the device name and vendor
  - For USB, PCI, matching is done either on the vendor ID/product ID, or the class → *grep* for these
- Driver file names are sometimes named in a "generic" way, not necessarily reflecting all devices they support.
  - Example: drivers/gpio/gpio-pca953x.c supports much more than just PCA953x. See the full list of devices supported by this driver



Finding Linux kernel drivers: an example

- You have a Maxim Integrated MAX7313 GPIO expander on I2C
- Search in the Linux kernel

```
$ git grep -i max7313
drivers/gpio/gpio-pca953x.c:    { "max7313", 16 | PCA953X_TYPE | PCA_INT, },
drivers/gpio/gpio-pca953x.c:    { .compatible = "maxim,max7313", .data = OF_953X(16, PCA_INT), },
```

- drivers/gpio/gpio-pca953x.c seems to support it
- Read drivers/gpio/Makefile to learn which kernel configuration option enables this driver:

```
obj-$(CONFIG_GPIO_PCA953X)                  += gpio-pca953x.o
```

- Conclusion: you need to enable CONFIG_GPIO_PCA953X in your kernel configuration



User-space interfaces to drivers



User-space interfaces for hardware devices

For a high-level perspective: three main interfaces to access hardware devices exposed by the Linux kernel

- Device nodes in `/dev`
- Entries in the *sysfs* filesystem
- Network sockets and related APIs



Devices in */dev/*

- One of the kernel important roles is to **allow applications to access hardware devices**
- In the Linux kernel, most devices are presented to user space applications through two different abstractions
  - **Character** device
  - **Block** device
- Internally, the kernel identifies each device by a triplet of information
  - **Type** (character or block)
  - **Major** (typically the category of device)
  - **Minor** (typically the identifier of the device)
- See Documentation/admin-guide/devices.txt for the official list of reserved type/major/minor numbers.



Block vs. character devices

- Block devices
  - A device composed of fixed-sized blocks, that can be read and written to store data
  - Used for hard disks, USB keys, SD cards, etc.
- Character devices
  - Originally, an infinite stream of bytes, with no beginning, no end, no size. The pure example: a serial port.
  - Used for serial ports, terminals, but also sound cards, video acquisition devices, frame buffers
  - Most of the devices that are not block devices are represented as character devices by the Linux kernel



Devices: everything is a file

- A very important UNIX design decision was to represent most *system objects* as files
- It allows applications to manipulate all *system objects* with the normal file API (`open`, `read`, `write`, `close`, etc.)
- So, devices had to be represented as files to the applications
- This is done through a special artifact called a **device file**
- It is a special type of file, that associates a file name visible to user space applications to the triplet *(type, major, minor)* that the kernel understands
- All *device files* are by convention stored in the `/dev` directory



Device files examples

Example of device files in a Linux system

```
$ ls -l /dev/ttyS0 /dev/tty1 /dev/sda /dev/sda1 /dev/sda2 /dev/sdc1 /dev/zero
brw-rw---- 1 root disk    8,  0 2011-05-27 08:56 /dev/sda
brw-rw---- 1 root disk    8,  1 2011-05-27 08:56 /dev/sda1
brw-rw---- 1 root disk    8,  2 2011-05-27 08:56 /dev/sda2
brw-rw---- 1 root disk    8, 32 2011-05-27 08:56 /dev/sdc
crw------- 1 root root    4,  1 2011-05-27 08:57 /dev/tty1
crw-rw---- 1 root dialout 4, 64 2011-05-27 08:56 /dev/ttyS0
crw-rw-rw- 1 root root    1,  5 2011-05-27 08:56 /dev/zero
```

Example C code that uses the usual file API to write data to a serial port

```c
int fd;
fd = open("/dev/ttyS0", O_RDWR);
write(fd, "Hello", 5);
close(fd);
```



Creating device files

- Before Linux 2.6.32, on basic Linux systems, the device files had to be created manually using the `mknod` command
  - `mknod /dev/<device> [c|b] major minor`
  - Needs root privileges
  - Coherency between device files and devices handled by the kernel was left to the system developer
- The `devtmpfs` virtual filesystem can be mounted on `/dev` → the kernel automatically creates/removes device files
  - CONFIG_DEVTMPFS_MOUNT → asks the kernel to mount *devtmpfs* automatically at boot time (except when booting on an initramfs).



Better handling of device files: *udev* and *mdev*

- *devtmpfs* is great, but its capabilities are limited, so complementary solutions exist
- **udev**
  - daemon that receives events from the kernel about devices appearing/disappearing
  - can create/remove device files (but that's done by *devtmpfs* now), adjust permission/ownership, load kernel modules automatically, create symbolic links to devices
  - according to rules files in `/lib/udev/rules.d` and `/etc/udev/rules.d`
  - used in almost all desktop Linux distributions
  - https://en.wikipedia.org/wiki/Udev
- **mdev**
  - lightweight implementation of *udev*, part of BusyBox
  - https://wiki.gentoo.org/wiki/Mdev



Examples of user-space interfaces in `/dev`

- Serial-ports: `/dev/ttyS*`, `/dev/ttyUSB*`, `/dev/ttyACM*`, etc.
- GPIO controllers (modern interface): `/dev/gpiochipX`
- Block storage devices: `/dev/sd*`, `/dev/mmcblk*`, `/dev/nvme*`
- Flash storage devices: `/dev/mtd*`
- Display controllers and GPUs: `/dev/dri/*`
- Audio devices: `/dev/snd/*`
- Camera devices: `/dev/video*`
- Watchdog devices: `/dev/watchdog*`
- Input devices: `/dev/input/*`
- and many more...



sysfs filesystem

- `block/`, symlinks to all block devices, in `/sys/devices`
- `bus/`, one sub-folder by type of bus
- `class/`, one sub-folder per class (category of devices): input, leds, pwm, etc.
- `dev/`
  - `block/`, one symlink per block device, named after major/minor
  - `char/`, one symlink per character device, named after major/minor
- `devices/`, all devices in the system, organized in a slightly chaotic way, see this article
- `firmware/`, representation of firmware data
  - `devicetree/`, directory and file representation of Device Tree nodes and properties
- `fs/`, properties related to filesystem drivers
- `kernel/`, properties related to various kernel subsystems
- `module/`, properties about kernel modules
- `power/`, power-management related properties



sysfs filesystem example

- `/sys/bus/i2c/drivers`: all device drivers for devices connected on I2C busses

```
[...]
edt_ft5x06
stpmic1
[...]
```

- `/sys/bus/i2c/devices`: all devices in the system connected to I2C busses

```
0-002a -> ../../../devices/platform/soc/40012000.i2c/i2c-0/0-002a
0-0039 -> ../../../devices/platform/soc/40012000.i2c/i2c-0/0-0039
0-004a -> ../../../devices/platform/soc/40012000.i2c/i2c-0/0-004a
1-0028 -> ../../../devices/platform/soc/5c002000.i2c/i2c-1/1-0028
1-0033 -> ../../../devices/platform/soc/5c002000.i2c/i2c-1/1-0033
i2c-0 -> ../../../devices/platform/soc/40012000.i2c/i2c-0
i2c-1 -> ../../../devices/platform/soc/5c002000.i2c/i2c-1
i2c-2 -> ../../../devices/platform/soc/40012000.i2c/i2c-0/i2c-2
```



sysfs filesystem example

/sys/bus/i2c/devices/0-002a/

```
lrwxrwxrwx    driver -> ../../../../../../bus/i2c/drivers/edt_ft5x06
-rw-r--r--    gain
drwxr-xr-x    input
-r--r--r--    modalias
-r--r--r--    name
lrwxrwxrwx    of_node -> ../../../../../../firmware/devicetree/base/soc/i2c@40012000/touchscreen@2a
-rw-r--r--    offset
-rw-r--r--    offset_x
-rw-r--r--    offset_y
drwxr-xr-x    power
-rw-r--r--    report_rate
lrwxrwxrwx    subsystem -> ../../../../../../bus/i2c
-rw-r--r--    threshold
-rw-r--r--    uevent
```

- `driver`, symlink to the driver directory in `/sys/bus/i2c/drivers`
- `of_node`, symlink to the directory for the Device Tree node describing this device



Example of driver interfaces in *sysfs*

- All devices are visible in *sysfs*, whether they have an interface in `/dev` or not
  - Usually `/dev` is to access the device
  - `/sys` is more about properties of the devices
- However, some devices only have a *sysfs* interface
  - LED: `/sys/class/leds`, see documentation
  - PWM: `/sys/class/pwm`, see documentation
  - IIO: `/sys/bus/iio`, see documentation
  - etc.



Accessing GPIOs

A class of devices worth mentioning is GPIOs (*General Purpose Input Output*)

- The GPIOs can be accessed through a legacy interface in `/sys/class/gpios`
  - You will find many instructions on the Internet about how to drive GPIOs through this interface.
  - However, this interface is deprecated and has multiple shortcomings:
    - GPIOs remain exported if the process using them crashes
    - Need to compute the GPIO numbers, such numbers are not stable
- A new interface recommended: libgpiod
  - Based on `/dev/gpiochipx` character devices
  - Implementing advanced features not possible with the legacy interface
  - Of course, this is a C library
  - But it also provides command line utilities: `gpiodetect`, `gpioset`, `gpioget`...
  - The only constraint is to cross-compile them for your target (the legacy interface could be used without any additional software).



Other virtual filesystems

- *debugfs*
  - Conventionally mounted in `/sys/kernel/debug`
  - Contains lots of debug information from the kernel, including device related
  - `/sys/kernel/debug/pinctrl` for pin-mux debugging, `/sys/kernel/debug/gpio` for GPIO debugging, `/sys/kernel/debug/pwm` for PWM debugging, etc.
  - https://www.kernel.org/doc/html/latest/filesystems/debugfs.html
- *configfs*
  - Conventionally mounted in `/sys/kernel/config`
  - Allows to manage configuration of advanced kernel mechanisms
  - Example: configuration of USB gadget functionalities
  - Documentation/filesystems/configfs.rst



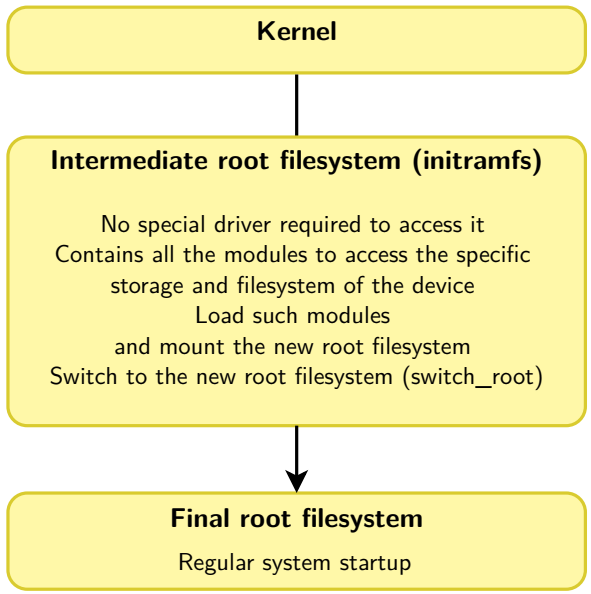
Using kernel modules



Why kernel modules?

- Primary reason: keep the kernel image minimal, and load drivers on-demand depending on the hardware detected
  - Needed to create a generic kernel configuration that works on many platforms
  - Used by all desktop/server Linux distributions
- But also useful for
  - Driver development: allows to modify, build and test a driver without rebooting
  - Boot time reduction: allows to defer the initialization of a driver after user-space has started critical applications

Using kernel modules to support many different devices and setups



The modules in the initramfs are updated every time a kernel upgrade is available.



Module installation and metadata

- As discussed earlier, modules are installed in `/lib/modules/<kernel-version>/`
- Compiled kernel modules are stored in `.ko` (*Kernel Object*) files
- Metadata files:
  - `modules.dep`
  - `modules.alias`
  - `modules.symbols`
  - `modules.builtin`
- Each file has a corresponding `.bin` version, which is an optimized version of the corresponding text file



Module dependencies: *modules.dep*

- Some kernel modules can depend on other modules, based on the symbols (functions and data structures) that they use.
- Example: the `ubifs` module depends on the `ubi` and `mtd` modules.
  - `mtd` and `ubi` need to be loaded before `ubifs`
- These dependencies are described both in `/lib/modules/<kernel-version>/modules.dep` and in `/lib/modules/<kernel-version>/modules.dep.bin`
- Will be used by module loading tools.

Module alias: *modules.alias*

Kernel compiling

```
static const struct usb_device_id products [] = {
{
        // Linksys USB200M
        USB_DEVICE (0x077b, 0x2226),
        .driver_info =  (unsigned long) &ax8817x_info,
}, {
        // Netgear FA120
        USB_DEVICE (0x0846, 0x1040),
        .driver_info =  (unsigned long) &netgear_fa120_info,
},
...
        { },            // END
};
MODULE_DEVICE_TABLE(usb, products);
```

drivers/net/usb/asix_devices.c

The device driver source code lists which devices it supports

make modules

Module file

asix.ko

Containing the list of supported devices (module metadata)

make modules_install (depmod)

```
alias usb:v077Bp2226d*dc*dsc*dp*ic*isc*ip*in* asix
alias usb:v0846p1040d*dc*dsc*dp*ic*isc*ip*in* asix
....
```

modules.alias

A new USB device is plugged-in

Thanks to aliases, the asix module is loaded

udev has a rule for when MODALIAS is set

System operation

The USB controller driver reads USB device attributes: vendor id, product id, etc

The USB bus driver sends a MODALIAS string encoding these attributes to the udev process

MODALIAS=usb:v077bp2226d0001dcFFdscFFdp00icFFiscFFip00in00

```
ENV{MODALIAS}=="?*", RUN{builtin}+="kmod load $env{MODALIAS}"
....
```

/lib/udev/rules.d/80-drivers.rules

kmod load usb:v077bp2226d0001dcFFdscFFdp00icFFiscFFip00in00



Module utilities: *modinfo*

- `modinfo <module_name>`, for modules in `/lib/modules`
- `modinfo /path/to/module.ko`

```
# modinfo usb_storage
filename:       /lib/modules/5.18.13-200.fc36.x86_64/kernel/drivers/usb/storage/usb-storage.ko.xz
license:        GPL
description:    USB Mass Storage driver for Linux
author:         Matthew Dharm <mdharm-usb@one-eyed-alien.net>
alias:          usb:v*p*d*dc*dsc*dp*ic08isc06ip50in*
alias:          usb:v*p*d*dc*dsc*dp*ic08isc05ip50in*
alias:          usb:v*p*d*dc*dsc*dp*ic08isc04ip50in*
[...]
intree:         Y
name:           usb_storage
[...]
parm:           option_zero_cd:ZeroCD mode (1=Force Modem (default), 2=Allow CD-Rom (uint)
parm:           swi_tru_install:TRU-Install mode (1=Full Logic (def), 2=Force CD-Rom, 3=Force Modem) (uint)
parm:           delay_use:seconds to delay before using a new device (uint)
parm:           quirks:supplemental list of device IDs and their quirks (string)
```



Module utilities: *lsmod*

- Lists currently loaded kernel modules
- Includes
  - The reference count: incremented when the module is used by another module or by a user-space process, prevents from unloading modules that are in-use
  - Dependent modules: modules that depend on us
- Information retrieved through `/proc/modules`

```
$ lsmod
Module                  Size  Used by
tun                    61440  2
tls                   118784  0
rfcomm                 90112  4
snd_seq_dummy          16384  0
snd_hrtimer            16384  1
wireguard              94208  0
curve25519_x86_64      36864  1 wireguard
libcurve25519_generic  49152  2 curve25519_x86_64,wireguard
ip6_udp_tunnel         16384  1 wireguard
```



Module utilities: *insmod* and *rmmod*

- Basic tools to:
  - *load* a module: `insmod`
  - *unload* a module: `rmmod`
- Basic because:
  - Need a full path to the module `.ko` file
  - Do not handle module dependencies

```
# insmod /lib/modules/`uname -r`/kernel/fs/fuse/cuse.ko.xz
# rmmod cuse
```



Module utilities: *modprobe*

- *modprobe* is the more advanced tool for loading/unloading modules
- Takes just a module name as argument: `modprobe <module-name>`
- Takes care of dependencies automatically, using the `modules.dep` file
- Supports removing modules using `modprobe -r`, including its no longer used dependencies

```
# modinfo fat_test | grep depends
depends:        kunit,fat
# lsmod | grep -E "^(kunit|fat|fat_test)"
fat                    86016  1 vfat
# modprobe fat_test
# lsmod | grep -E "^(kunit|fat|fat_test)"
fat_test               24576  0
kunit                  36864  1 fat_test
fat                    86016  2 fat_test,vfat
# sudo modprobe -r fat_test
# lsmod | grep -E "^(kunit|fat|fat_test)"
fat                    86016  1 vfat
```



Passing parameters to modules

- Some modules have parameters to adjust their behavior
- Mostly for debugging/tweaking, as parameters are global to the module, not per-device managed by the module
- Through `insmod` or `modprobe`:

```
insmod ./usb-storage.ko delay_use=0
modprobe usb-storage delay_use=0
```

- `modprobe` supports configuration files: `/etc/modprobe.conf` or in any file in `/etc/modprobe.d/`:

```
options usb-storage delay_use=0
```

- Through the kernel command line, when the module is built statically into the kernel:

```
usb-storage.delay_use=0
```

  - `usb-storage` is the *module name*
  - `delay_use` is the *module parameter name*. It specifies a delay before accessing a USB storage device (useful for rotating devices).
  - `0` is the *module parameter value*



Modules in *sysfs*

- All modules are visible in *sysfs*, under `/sys/module/<name>`
- Lots of information available about each module
- For example, the `/sys/module/<name>/parameters` directory contains one file per module parameter
- Can read the current value of module parameters
- Some of them can even be changed at runtime (determined by the module code)
- Example: `echo 0 > /sys/module/usb_storage/parameters/delay_use`